Laboratorio de Datos

TRABAJO PRÁCTICO 02

Grupo : "NULL"

Lucía Murillo - Victoria Florencia Lima -
María Cecilia Marchesano - Emma Tripaldi

Introducción

En el presente trabajo abordamos un problema de aprendizaje automático supervisado: a partir de imágenes con su clase conocida, buscamos construir un modelo que, dada una nueva imagen, prediga a qué clase pertenece. Los datos provienen del juego Quick, Draw! de Google [1], en el que millones de personas dibujan distintos objetos. Trabajamos con 30000 imágenes en escala de grises de 28 x 28 píxeles, distribuidas en 10 clases distintas (como círculo, casa o pez).
Para llegar al modelo final, primero fue necesario explorar el dataset de distintas formas para conocer sus características. Luego nos enfocamos en dos clases específicas (cuadrado y casa) y probamos modelos de clasificación utilizando k vecinos más cercanos (KNN), con el fin de elegir píxeles estratégicos a la hora de reconocer una imagen. Finalmente, abordamos el problema de clasificación multiclase utilizando árboles de decisión, comparando distintas configuraciones de hiperparámetros mediante validación cruzada para decidir cuál era el mejor modelo. En cada caso, evaluamos el desempeño con las métricas que mejor se ajustaban a las características del problema. El modelo elegido se evaluó en un conjunto *held-out,* separado desde un principio, que no fue utilizado ni para seleccionar el árbol ni para entrenarlo, y a partir de su desempeño en ese conjunto estimamos la accuracy esperada sobre las imágenes proporcionadas por la cátedra.

Análisis exploratorio de los datos

Descripción del conjunto de datos

Para empezar el análisis exploratorio, lo primero que hicimos fue ver cómo estaba compuesto el dataset. Cada fila es un dibujo descrito por 785 atributos, que son los píxeles de una grilla de 28 x 28, junto con la clase a la que pertenece. Como tantos números sueltos no dicen nada sobre cómo es un dibujo, decidimos graficar uno de ejemplo (Figura 1). Cada píxel es un entero entre 0 (sin trazo) y 255 (trazo más intenso posible), y no hay valores faltantes.
Observamos que tenemos 30000 imágenes repartidas entre 10 clases. Mediante una consulta SQL contamos cuántas había de cada una (Tabla 1, que también indica a qué objeto corresponde cada una), y vimos que las clases están casi balanceadas: van de 2870 a 3120 imágenes.

Cantidad de imagenes de cada clase

label	objeto	cantidad_dibujos
6	estrella	3120
2	cuadrado	3085
8	remera	3065
0	circulo	3040
4	triangulo	3010
7	paraguas	2990
3	pez	2960
9	casa	2945
1	escalera	2915
5	colectivo	2870

Tabla 1: cantidad de dibujos por clase, ordenada en forma descendente por cantidad .

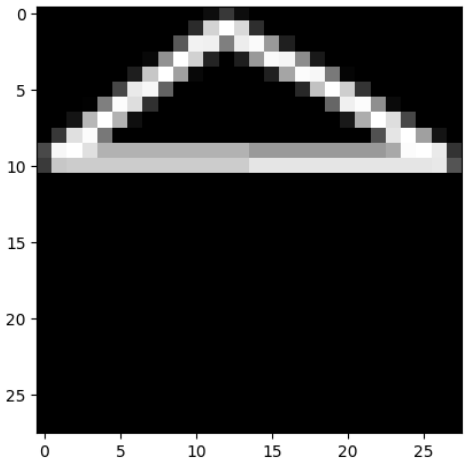


Figura 1: dibujo exploratorio para ver distintas intensidades de los píxeles.

1.a. Relevancia de los atributos.

Para analizar qué píxeles pueden ser relevantes, se nos ocurrió hacer un mapa de calor con la intensidad media de cada píxel sobre todas las imágenes (Figura 2). A simple vista se observa que los bordes de la grilla, sobre todo en la esquina inferior derecha, están mucho más oscuros: en muy pocos dibujos hay trazo allí, por lo que esos píxeles aportan poca información y potencialmente podrían ser descartados. En cambio, la mayor intensidad promedio se encuentra en la parte superior, debajo del margen, y el lado izquierdo de la imagen.

Sin embargo, que un píxel tenga una intensidad promedio baja no alcanza para descartarlo: lo importante es que su valor varíe de una clase a otra. Como vemos en los mapas de calor por clase (Figura 3, inciso siguiente), cada una utiliza zonas relevantes distintas, por lo que un píxel poco utilizado en general puede ser clave para separar dos clases puntuales. En definitiva, pensamos que los píxeles de los márgenes podrían descartarse, pero la relevancia de un atributo depende de las clases que se quieran distinguir, y por eso conviene elegirlos según el problema.

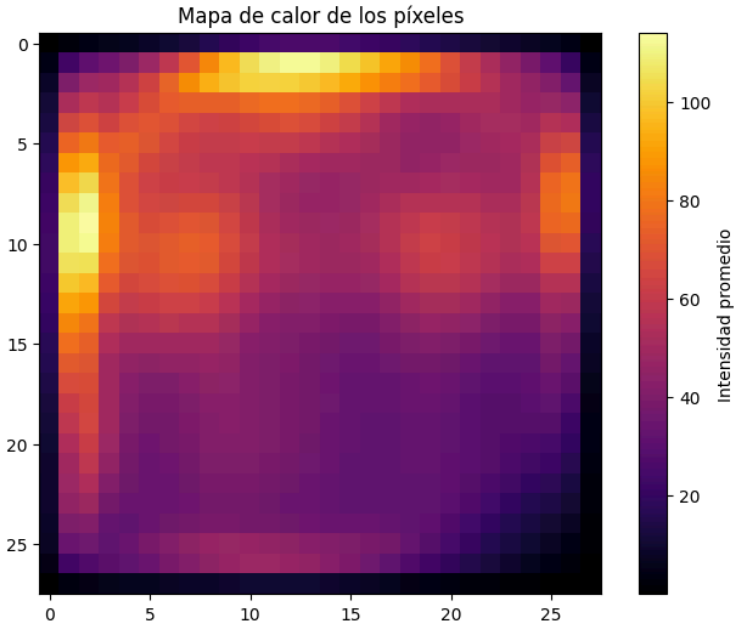


*Figura 2: Mapa de calor del promedio de intensidad por pixel de **todas** las clases.*

1.b. Similitud entre clases.

A fin de observar las similitudes entre las clases, graficamos el mapa de calor de cada una de ellas por separado (Figura 3). Observamos, por ejemplo, que el pez y el colectivo podrían ser difíciles de diferenciar entre sí, ya que sus píxeles de mayor intensidad se encuentran en la misma zona.

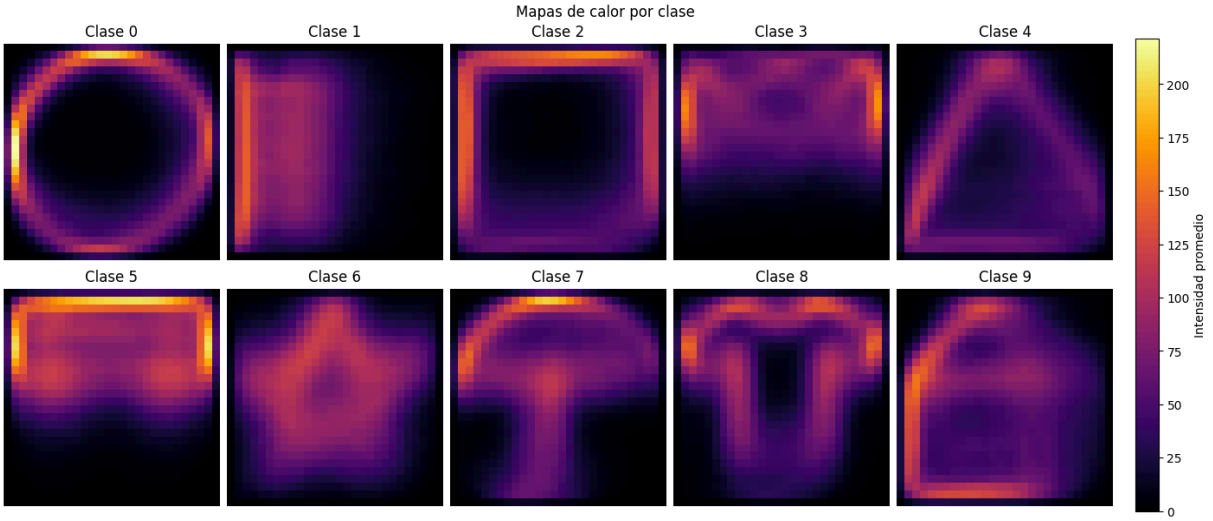


Figura 3: Mapa de calor de cada una de las clases (realizado con el promedio por pixel de todas las imágenes de la clase correspondiente).

Profundizando en las similitudes entre pez y colectivo, graficamos la diferencia de intensidad media de cada píxel en ambas clases, la elevamos al cuadrado y la ordenamos de mayor a menor (el eje x de la Figura 4 no representa el número de los píxeles). Como referencia, hicimos lo mismo con el pez y el círculo, una clase con forma muy distinta (aro con el centro vacío). Observamos que la diferencia entre el pez y el colectivo (en azul) es más chica que entre el pez y el círculo (en rojo): en el primer caso tiende más rápido a cero y también se aplana mucho antes, es decir, hay menos píxeles donde difieren.

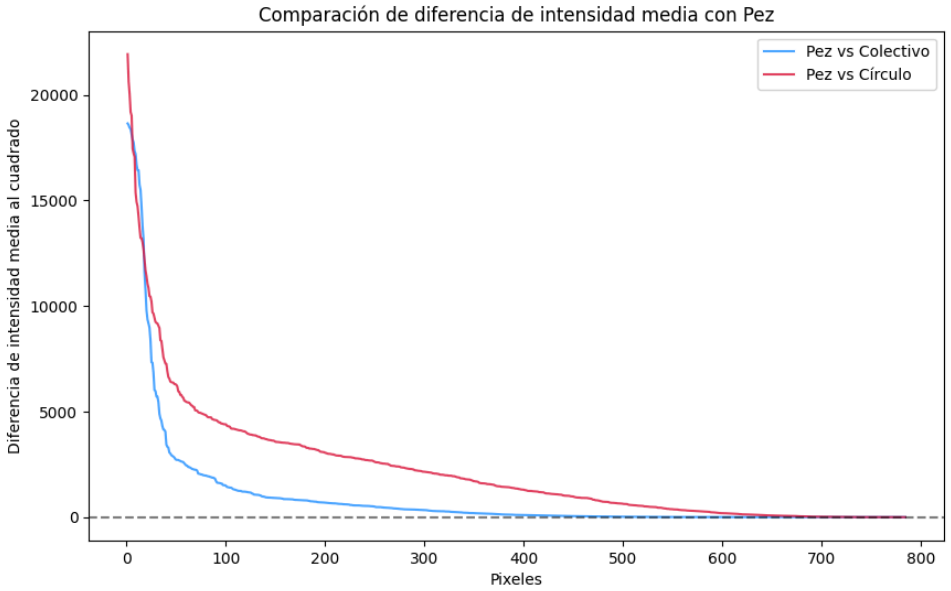


Figura 4: comparación de la diferencia de intensidad media por píxel (al cuadrado) entre pez y colectivo, y entre pez y círculo, ordenada de mayor a menor.

1.c. Dispersión dentro de una clase.

Para ver qué tan parecidas son las imágenes dentro de una misma clase, tomamos la clase 3 (pez). Como primer acercamiento, graficamos diez imágenes para tener ejemplos concretos, teniendo en cuenta que la muestra es chica y solo podemos hacer conjeturas a partir de ella.
Como se observa en la Figura 5, los dibujos de peces pueden ser muy diferentes entre sí: varían en orientación (algunos tienen la cola a la derecha y otros a la izquierda, o están torcidos), en tamaño, en los detalles (ojos, escamas, aletas) y en la forma del cuerpo.

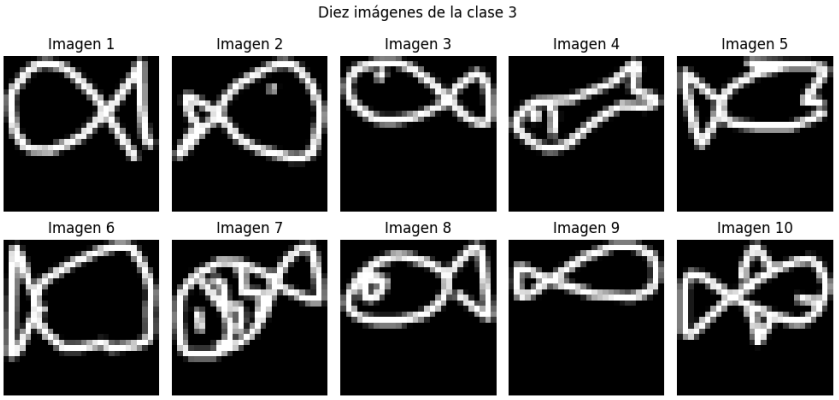


Figura 5: primeras diez imágenes de la clase 3 (pez).

Para cuantificar esta variación, calculamos la distancia euclídea entre cada imagen y la imagen promedio de su clase, restando píxel a píxel. Así, cada dibujo queda resumido en un número que refleja cuánto se aleja del dibujo típico de su clase. Comparamos esta distribución para las clases 0 (círculo), 3 (pez) y 8 (remera) graficandolas en un boxplot (*Figura 6*). Observamos que el 50% central de los datos (entre percentiles 25 y 75) abarca un rango parecido en las tres clases, y todas muestran una dispersión considerable con múltiples valores atípicos, en particular el pez y el círculo, con imágenes que llegan a distancias de más de 3000, mientras que en la remera los outliers no pasan de 2800 aproximadamente. Incluso el círculo, que esperábamos que sea más homogéneo, tiene dibujos muy alejados del promedio, aunque presenta la menor mediana, lo cual es esperable en una figura simple.
Concluimos entonces que las imágenes de una misma clase no son muy similares entre sí, y que esto no ocurre solo en el pez. Cabe aclarar que una limitación de este análisis es que el boxplot resume cada imagen en un único valor (generando que dos dibujos con la misma cantidad de tinta pero en lugares distintos puedan dar distancias grandes), pero no distingue en qué zona difiere cada dibujo.

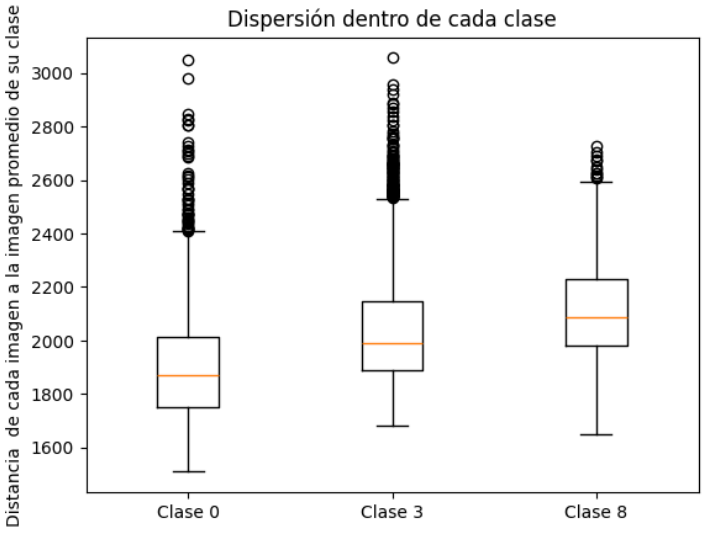


Figura 6: boxplot de la dispersión dentro de cada clase (círculo, pez y remera respectivamente).

1.d. Dificultad sobre imágenes

Consideramos que la naturaleza del dataset, compuesto por imágenes, representa en parte una complicación en la exploración de los datos. Por un lado, a diferencia de Titanic, donde cada columna tiene significado propio (edad, clase, sexo), acá cada atributo es un píxel que por sí solo no se puede interpretar. Además, al ser 784 atributos en lugar de unos pocos, se vuelve inviable analizar variable por variable con histogramas o tablas y mirar los números del archivo no aporta nada. Es por esto que la forma más útil de explorar los datos fue graficarlos.
Por otro lado, hay ventajas: todos los atributos están en la misma escala y no hay valores faltantes. Una vez que se entiende la estructura de píxeles vecinos que forman la imágen, herramientas como los mapas de calor y las imágenes promedio permiten explorar el dataset de forma muy intuitiva.

Clasificación binaria

2.a. Análisis del nuevo dataset de clases 2 y 9

Para empezar, el dataframe original está relativamente balanceado ya que la clase que más dibujos tiene es la 6 con 3120, mientras que la que menos tiene es la 5 con 2870 imágenes. Luego, es esperable que ocurra lo mismo en el caso de las clases 2 y 9 (cuadrado y casa). Para ser exactas, hay una diferencia de 140 imágenes entre estas dos clases (a favor del cuadrado) que representa el 2% del total en estas dos clases. Es por esto que el dataframe está balanceado.

2.b. Elección métricas de evaluación

Partiendo de la base de que el subconjunto de datos está balanceado, elegimos Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$.
Además, luego de debatir, llegamos a la conclusión de que no es lo mismo cómo se equivoca el modelo al predecir la clase: es más problemático que el modelo considere que el objeto es un cuadrado pero se trate de una casa, es decir, que ignore todos los detalles adicionales de una clase más “sofisticada” como la 9. Para ello, establecimos que el cuadrado era el valor positivo, mientras que la casa era el valor negativo, y así se convirtió en un problema de Tipo I. En este sentido, un falso positivo es una casa que aparenta ser un cuadrado, mientras que un falso negativo es un cuadrado que aparenta ser una casa. Es por esto que escogimos la métrica de Precision ya que en su fórmula $\frac{TP}{TP + FP}$ aparecen los FP en el denominador y a menor cantidad de los mismos, menor denominador y por ende mayor número total, es decir precisión. Como consecuencia, un modelo con pocos FP (casos graves) significaría un modelo mejor.

2.c. Separacion datos

Con el objetivo de que los conjuntos de entrenamiento y evaluación quedaran bien divididos y las clases quedaran distribuídas equitativamente, a la función “train_test_split” de sklearn le pasamos el parámetro stratify que toma una lista con las clases o labels a equiparar.

2.d. Modelo de KNN con cantidad reducida de atributos

Para decidir con qué píxeles trabajar, ordenamos de manera descendente a los 784 según cuánto se diferencia su valor promedio en las casas y en los cuadrados, utilizando únicamente los datos de

entrenamiento. A partir de dicho ranking, elegimos conjuntos de 2, 3 y 5 atributos tomados de distintas zonas: los primeros, los bloques que les siguen, un grupo alrededor del puesto 100, los últimos (esperando que este sea el que peor rinda), y uno elegido al azar. Para cada conjunto, ajustamos un modelo de KNN con k = 5 sobre los datos de entrenamiento y evaluamos su desempeño en los datos de testeo utilizando las dos métricas elegidas, accuracy y precision.

Observamos (en la Tabla 3) cómo los últimos píxeles del ranking rinden como el azar (accuracy del 0.51, que es la proporción de cada clase), mientras que los que están entre los primeros 100 van de 0.74 a 0.90. No obstante, los primeros del ranking no son los mejores, ya que hasta el sexto píxel se quedan en 0.83 aproximadamente, mientras que los puestos del 6 al 8, con solo 3 píxeles, llegan a 0.898 de accuracy y 0.936 de precision.

Para entender el motivo de este fenómeno, ubicamos estos dos grupos de píxeles en una matriz de 28x28, simulando las imágenes. Como se observa en la *Figura 7*, los primeros tres del ranking están uno al lado del otro, por lo que aportan información repetida al modelo, mientras que los del 6 al 8 están separados y se ubican en lugares más estratégicos, en un lateral y hacia una de las esquinas superiores, donde en una casa no suele haber trazo (el techo deja este espacio) y en un cuadrado sí.

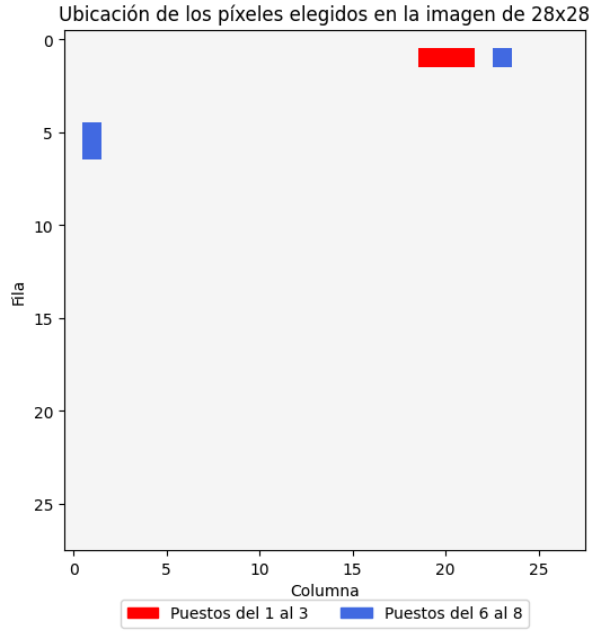


Figura 7: Ubicación de los píxeles elegidos de la imagen.

Ranking de la diferencia de la media por píxeles

pixel48	148.654529
pixel47	147.124656
pixel49	144.969134
pixel46	141.584329
pixel50	137.687022
pixel45	133.112859
pixel141	129.283314
pixel169	128.204968
pixel51	126.338810
pixel113	124.387661

Tabla 2: primeras diez líneas del ranking de la diferencia de píxeles.

Por otro lado, como se observa en la Tabla 3, resulta claro que la precision separa conjuntos que la accuracy iguala, por ejemplo con puestos 100 al 103 y los primeros 3 tienen misma accuracy (0.837), pero su precision varía contundentemente (0.773 y 0.904 respectivamente), es decir, el segundo grupo mencionado comete menos falsos positivos, o casas que pasaron como cuadrados. En cuanto a la cantidad de atributos, sumar píxeles ayuda cuando los de partida tienen peor desempeño (los puestos 100 en adelante pasan de 0.742 a 0.837 y a 0.879 con 2, 3 y 5 píxeles), pero casi no cambia cuando ya predecían bien (los primeros se mantienen alrededor de 0.835).

Resultados prediccion clase 2 y 9 con KNN, k = 5

	atributos	cantidad atributos	accuracy	precision
0	del 6 al 8	3	0.898	0.936
1	del 5 al 10	5	0.891	0.902
2	del 10 al 15	5	0.890	0.916
3	del 100 al 105	5	0.879	0.861
4	del 3 al 6	3	0.847	0.891
5	del 1 al 3	3	0.837	0.904
6	del 1 al 5	5	0.837	0.888
7	del 100 al 103	3	0.837	0.773
8	del 1 al 2	2	0.833	0.898
9	del 3 al 4	2	0.825	0.872
10	del 5 al 6	2	0.823	0.850
11	5 random	5	0.767	0.744
12	del 100 al 102	2	0.742	0.671
13	3 random	3	0.621	0.583
14	2 random	2	0.611	0.577
15	ultimos 5	5	0.536	0.536
16	ultimos 2	2	0.511	0.513
17	ultimos 3	3	0.510	0.514

Tabla 3: desempeño de cada KNN para las clases 2 y 9, con k=5, ordenada por mayor accuracy y precisión

2.e. Selección del modelo de KNN final

Para elegir el modelo final variamos el valor de vecinos (k = 2, 3, 4, 5, 10 y 50) y entrenamos al modelo con cinco conjuntos de atributos de distintas cantidades. Observamos en la Tabla 4 que el valor de k no modifica demasiado el rendimiento sobre los datos de testeo, sino que lo que más pesa son los píxeles con los que se entrenó. Los primeros 9 píxeles lograron la mayor accuracy (0.918 con k = 10), pero los puestos 6 al 8 alcanzan 0.90 con solo tres atributos y la mayor precision de la tabla (0.944 con k = 3), métrica que consideramos importante en el inciso b. Considerando el costo computacional que conlleva utilizar el triple de atributos y mayor k para obtener un resultado levemente mejor, elegimos el conjunto de píxeles de los puestos 6 a 8 del ranking como el final. Vale aclarar que, como comparamos el modelo sobre el mismo test, los resultados son optimistas, razón por la que en el ejercicio 3 se utilizará validación cruzada y un conjunto held out.
Modelo elegido de KNN: píxeles 141, 169 y 51, con k = 3.

Comparación modelos KNN predicción clase 2 y 9

	atributos	cantidad atributos	K	accuracy	precision
0	primeros 9	9	10	0.918	0.927
1	primeros 9	9	5	0.911	0.926
2	primeros 9	9	4	0.911	0.911
3	primeros 9	9	50	0.909	0.935
4	primeros 9	9	3	0.905	0.924
5	primeros 9	9	2	0.905	0.897
6	del 6 al 8 (elegido)	3	10	0.902	0.938
7	del 6 al 8 (elegido)	3	50	0.900	0.943
8	del 6 al 8 (elegido)	3	3	0.898	0.944
9	del 6 al 8 (elegido)	3	5	0.898	0.936
10	del 6 al 8 (elegido)	3	4	0.896	0.923

Tabla 4: primeras once líneas del desempeño de cada KNN, variando también el valor de k, ordenado por mayor accuracy

Clasificación multiclase

3.a. Separación de los datos en dev y held-out

Separamos las 30000 imágenes en un conjunto de desarrollo (*dev* 80 %, 24000 imágenes) y uno de control (*held-out* 20 %, 6000 imágenes), utilizando la función “train_test_split” con estratificación por clase y semilla fija (21). La estratificación asegura que cada clase mantenga en ambos conjuntos la misma proporción que en el dataset original (aproximadamente 10 %), y la semilla hace que la partición sea reproducible. A continuación, y hasta el inciso d, dejamos apartadas las imágenes del held-out, así todas las decisiones que tomemos sobre el modelo no estarán condicionadas por estos datos.

3.b. Elección de métricas

Para poder cuantificar qué tan bien funciona nuestro modelo necesitamos elegir métricas, teniendo en cuenta los distintos errores que puede cometer y qué peso le damos a cada uno. Como vimos en el EDA, las clases están prácticamente balanceadas, así elegimos accuracy sabiendo que no se distorsionará por una mayoritaria. Además, ninguna clase es más importante que otra, por lo que consideramos que todos los errores son igual de importantes.

3.c. Comparación de árboles

Para comparar árboles de decisión utilizamos validación cruzada de 10 folds sobre el conjunto de desarrollo (en cada iteración 21600 imágenes para entrenar y 2400 para validar), haciendo uso nuevamente del parámetro “stratify” para conservar la proporción de clases por fold. Variamos tres hiperparametros de los árboles: la profundidad máxima (2, 5, 7 y 10), el criterio de corte (gini o entropía) y min_samples_leaf para imponer la cantidad de imágenes que deben entrar en cada hoja (1, el valor por defecto, o 5).
Realizamos, para cada configuración de hiper parámetros, 10 entrenamientos testeando con el conjunto correspondiente y midiendo su accuracy, para luego tomar el promedio de los 10 valores como el desempeño del modelo. En la Tabla 5 se distingue el desempeño de cada una de las configuraciones.

Resultados accuracy clasificación con arboles binarios

	hiperparametros	prof 2	prof 5	prof 7	prof 10
0	gini	0.2921	0.6355	0.7195	0.7787
1	entropia	0.3065	0.6280	0.7330	0.7821
2	gini con min_samples_leaf	0.2921	0.6355	0.7193	0.7780
3	entropia con min_samples_leaf	0.3065	0.6280	0.7329	0.7815

Tabla 5: accuracy de cada árbol

Observando la tabla podemos ver que la profundidad es lo que más impacta en el desempeño del modelo, el accuracy pasa de ≈ 30 a ≈78 cuando su profundidad sube de 2 a 10. Con profundidad 2 el árbol tiene a lo sumo 4 hojas, por lo que puede predecir a lo sumo 4 de las 10 clases . Por otro lado, el criterio casi no influye, la entropía es mejor que gini en 3 de las 4 profundidades (2, 7 y 10), pero su diferencia es de unos pocos decimales. En cuanto al hiper parámetro min_samples_leaf con valor 5 no mejora la métrica, en profundidades 2 y 5 los resultados son iguales y en 7 y 10 son apenas peores. También observamos que la mejor profundidad es con su valor máximo permitido, lo cual sugiere que un árbol más profundo rendiría más.

La mejor configuración y por lo tanto la elegida, fue con entropía, profundidad 10 y min_samples_leaf por defecto, con una accuracy promedio de 0,7821. A continuación (Figura 8), y en relación al estudio del inciso b sobre los posibles errores del modelo, presentamos la matriz de confusión elaborada a partir de todos los datos de la validación cruzada. Se observa cómo los elementos de la diagonal representan la cantidad de imágenes correctamente clasificadas para cada clase, y al sumarlas y dividirlas por el total de imágenes se obtiene el valor de la accuracy presentado anteriormente. Si bien habíamos elegido solamente la métrica de accuracy, la matriz muestra que el modelo no falla por igual en todas las clases. Las mejor clasificadas son el triángulo y el círculo, y las peor clasificadas son la estrella, la casa y el pez.

```
Matriz de confusión de los datos de entrenamiento
[[2071   15  118   42   24   16    6   33   60   47]
 [  10 1895   37   25   76    6  162   25   26   70]
 [ 147   48 1963   52   29   60   19   20   81   49]
 [  53   33   74 1675   33  142   82   98  143   35]
 [  14   56   10   25 2098    0  102   29   15   59]
 [  16    5   70  142    7 1866   11   67   86   26]
 [   9  153   18   84  142   13 1808   94   89   86]
 [  38   19   21   90   61   86   81 1862   91   43]
 [  35   17   55  120   32   94   60  150 1834   55]
 [  88   62   42   61  142   19   95   98   50 1699]]
```

Figura 8: Matriz de confusión

3.d. Entrenamiento y predicción held-out

En este punto utilizamos todo el conjunto de desarrollo (24000 imágenes) para entrenar un árbol con la configuración elegida en el inciso previo (entropía, profundidad 10, min_samples_leaf por defecto) y lo usamos para predecir las clases del conjunto held-out (6000 imagenes), que no se había usado hasta el momento. La accuracy fue de 0.7923 (4754 de 6000 imágenes bien clasificadas).
El resultado es coherente con el de la estimación de la validación cruzada (0.7821) y era esperable que sea mayor, pues en este inciso estamos entrenando al modelo con todos los datos de desarrollo (sin dejar de lado particiones).

3.e. Estimación del desempeño

Finalmente, utilizamos el modelo seleccionado para predecir la clase de cada imagen del conjunto de evaluación de la cátedra, cuyas etiquetas no conocemos, y guardamos estas predicciones en el archivo "*TP-02-0.7923-NULL.csv*".
Teniendo en cuenta que no conocemos las clases reales de estas imágenes no podemos precisar el desempeño de nuestro modelo en este conjunto, pero a partir del resultado del inciso anterior, estimamos una accuracy de 0.7923 (medida en el held-out). Consideramos que este valor representa bien lo que ocurrirá con imágenes que el modelo nunca vio, ya que el conjunto con el que lo evaluamos fue separado al comienzo, y no participó del entrenamiento ni de la elección de hiperparámetros y para este inciso no entrenamos al modelo adicionando esos datos. De esta manera, para el modelo esas 6000 imágenes son datos nuevos, al igual que las del conjunto de la cátedra.
Por último, esperamos que la accuracy real sea parecida a la obtenida en la validación cruzada (0.7821). Si bien el modelo final se entrenó con más datos (24000 en vez de 21600 imágenes), lo cual podría mejorar su desempeño levemente, la validación cruzada se usó para elegir la configuración, por lo que su resultado puede ser algo optimista. Como ambos efectos son pequeños, esperamos que se compensen.

Conclusiones

En este trabajo abordamos el problema de clasificación binaria y multiclase de imágenes del dataset de Quick, Draw! con modelos de aprendizaje supervisado.
En cuanto a la clasificación binaria, luego de experimentar con diferentes modelos de KNN para la distinción entre las clases casa y cuadrado, la elección de los atributos resultó ser significativamente más determinante que el valor de k. Llegamos a un modelo que con tan solo 3 píxeles bien elegidos alcanzó una accuracy de 0.898 y una precision de 0.944. Dado que el entrenamiento y comparación de modelos de este punto se hizo sobre el mismo conjunto con el que se testeó, reconocemos que el resultado tiene un sesgo optimista.
Para el caso de clasificación multiclase, comparamos configuraciones de árboles de decisión binaria para distinguir imágenes de 10 clases. En este punto la profundidad máxima del árbol fue el hiperparámetro que más influyó en los resultados. El árbol con mejor desempeño obtenido fue el de entropía y profundidad 10, alcanzando una accuracy de 0.7821 en el conjunto de desarrollo mediante validación cruzada y 0.7923 en el conjunto de evaluación (held-out). El análisis de la matriz de confusión evidenció que las clases que más le costó distinguir fueron pez, casa y estrella.
Teniendo en cuenta que el árbol elegido fue entrenado sobre todo el conjunto de desarrollo, estimamos un desempeño con 0.7923 de accuracy para las predicciones del conjunto de evaluación de la cátedra, entregadas en el archivo "*TP-02-0.7923-NULL.csv*". A diferencia de los resultados de clasificación binaria, consideramos que esta es una métrica sin sesgos y representativa del desempeño del modelo frente a datos desconocidos, ya que fue obtenida al evaluar en las predicciones del conjunto de held-out, que no participó del entrenamiento del árbol ni de la variación de hiperparametros.

Referencias

[1] Ha, David, and Douglas Eck. "A Neural Representation of Sketch Drawings." arXiv preprint arXiv:1704.03477 (2017).